

1. Context and Architecture of the RAG Evaluation Suite

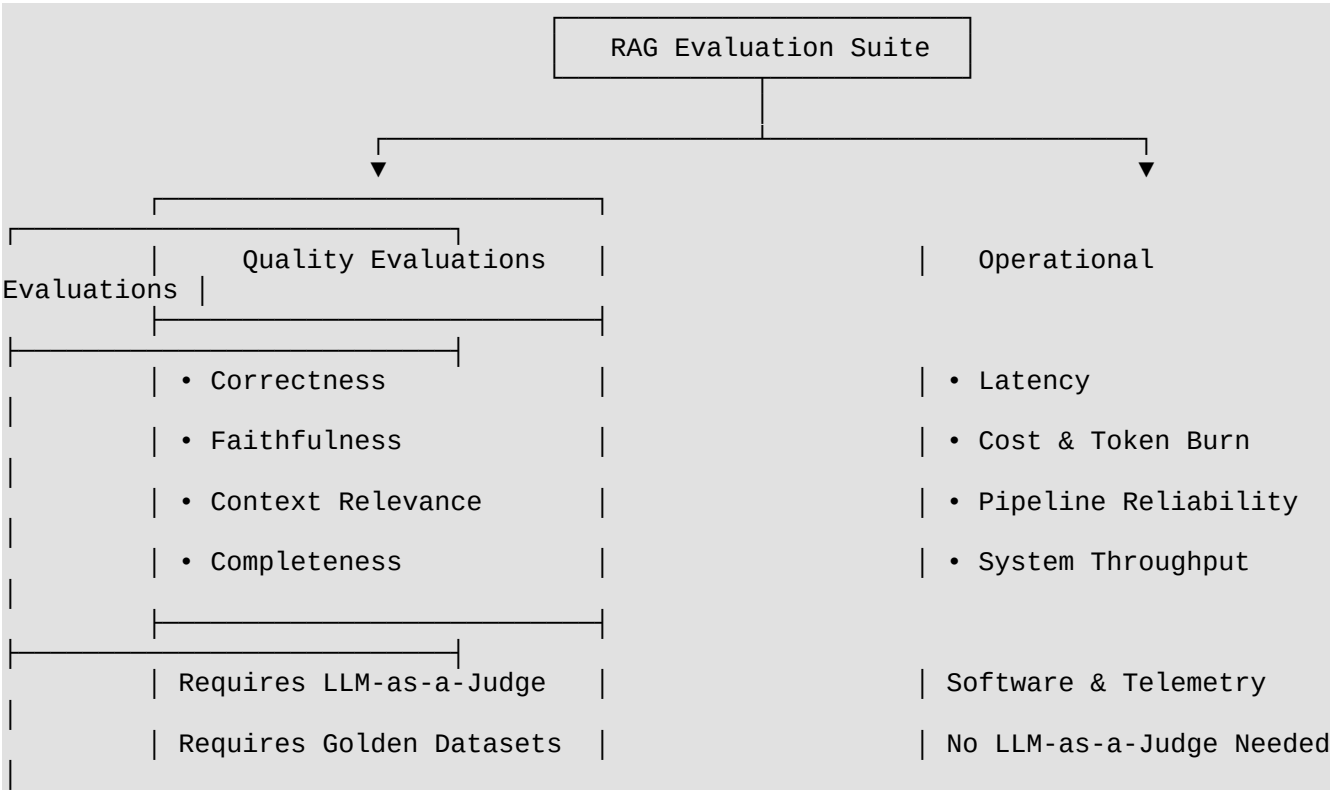
Positioning Operational Evals in the Evaluation Framework

In our roadmap for building a comprehensive Retrieval Augmented Generation (RAG) evaluation suite, we have established a structured three level framework comprising component level evaluations, pipeline level evaluations, and application- evel evaluations. Our concrete evaluation target throughout this series is a doubt solving chatbot designed to assist students enrolled in our AI/LLM course. Having previously completed the component level, pipeline level, and application level quality and safety evaluations, the final remaining module in our flow chart is operational evaluations.

2. Fundamental Principles of Operational Evaluations

Definition and Core Objective

Operational evaluations for RAG applications answer a fundamentally distinct question compared to quality evaluations. While quality evaluations assess outputs using metrics such as correctness, faithfulness, relevance, and completeness, operational evaluations evaluate whether the system operates sustainably in a production environment. Specifically, operational evaluations answer a critical question, even if the RAG system yields accurate answers, can it run **reliably**, **quickly**, and **economically** in production at scale?



Key Differences: Operational Evals vs. Quality Evals

Operational evaluations differ from quality evaluations across several technical dimensions:

- **Evaluation Driving Mechanism:** Quality evaluations rely primarily on LLMs operating as judges to critique output text. Operational evaluations are strictly **software driven and telemetry driven** measurement procedures executed directly against system code.
- **Resource Prerequisites:** Quality evaluations require curated golden datasets and LLM API calls for evaluation. Operational evaluations **do not require an LLM as a judge** and **do not require golden datasets**.
- **Execution Cost:** Quality evaluation scripts incur substantial LLM API costs to judge answers. Operational evaluation scripts are **completely free to run** because they only log software timings and token counts without invoking external LLM judge models.

The Primary Operational Metrics

Our operational evaluation framework measures three primary pillars:

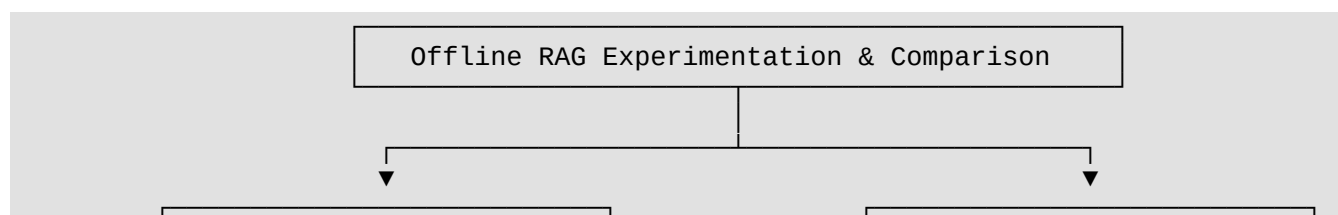
1. **Latency:** The duration users must wait before receiving generated responses.
2. **Cost:** The monetary expenditure required to process and answer each user query.
3. **Reliability:** The frequency with which the pipeline completes execution successfully versus encountering errors, timeouts, or pipeline failures.

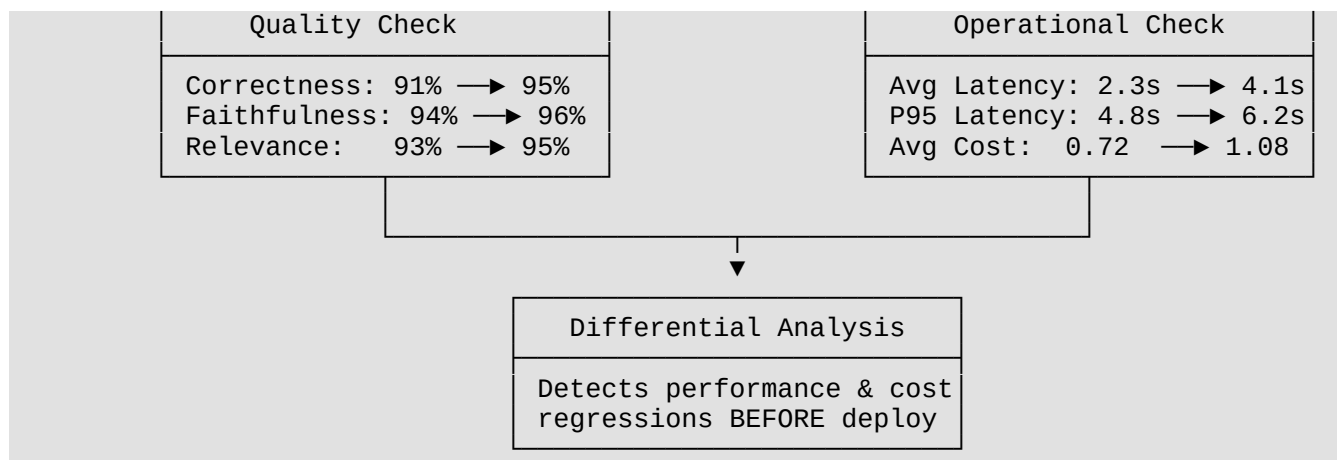
A fourth metric, **Throughput** (the volume of concurrent requests a system can process per unit of time), requires stress and load testing tools that fall outside our current single system scope, making Latency, Cost and Reliability our primary focus today.

3. The Necessity of Offline Operational Evals (Pre Deployment)

The Post Deployment Misconception

Because operational evaluations measure production runtime behaviors, it is often assumed that they belong exclusively in post deployment monitoring. However, omitting operational evaluations from our offline evaluation suite prior to deployment is a critical mistake. Offline operational evaluations are essential to prevent deploying pipelines that are impractically slow or prohibitively expensive.





Comparative Empirical Case Study: System Modifications

To demonstrate why offline operational evaluations are required, consider a controlled baseline experiment and a subsequent pipeline optimization:

- **Baseline RAG Pipeline:**
 - **Architecture:** Retrieves top-5 chunks via vector search and passes them directly to the generator LLM.
 - **Offline Quality Metrics:** Correctness = 91%, Faithfulness = 94%, Answer Relevance = 93%.
 - **Offline Operational Metrics:** Average Latency = 2.3 seconds, P95 Latency = 4.8 seconds, Average Cost per Query = 0.72 INR (72 paise), Timeout Rate = 2.0%, Success Rate = 99.8%.
- **Modified RAG Pipeline:**
 - **Architecture Changes:** Integrated a cross encoder re ranker, increased context retrieval from top-5 to top-10 chunks, and swapped the generator for a larger LLM.
 - **Offline Quality Metrics:** Correctness rose to 95% (+4%), Faithfulness rose to 96% (+2%), Answer Relevance rose to 95% (+2%).
 - **Offline Operational Metrics:** Average Latency jumped to 4.1 seconds (+78%), P95 Latency jumped to 6.2 seconds (+29%), Average Cost jumped to 1.08 INR (+50%), Timeout Rate dropped to 1.0%, Success Rate reached 99.9%.

If we only tracked quality metrics, we would have deployed the modified pipeline based on its quality gains. However, doing so would blindside us in production with severe latency increases and cost overruns.

Absolute Values vs. Differential Directions

While absolute latency values measured on a local development laptop (e.g., 2.3 seconds) will not match production server latency exactly, the **differential direction** (e.g., a shift from 2.3 seconds to 4.1

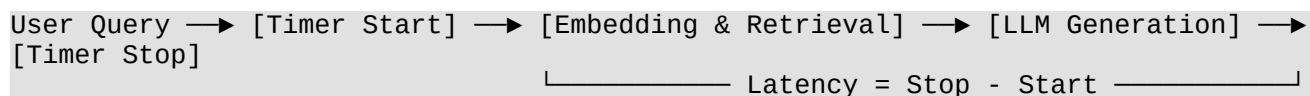
seconds) remains accurate and informative. Studying these directional shifts offline alerts engineering teams to performance regressions whenever pipeline parameters are modified. The foundational rule for RAG engineering is: **Do not wait until production to discover that your RAG pipeline is too slow or too expensive.**

4. Latency Evaluation

Definition and Core Mechanics

Latency is defined as the total duration a system requires to respond to a user request specifically, the elapsed time between a user submitting a prompt and the complete response displaying on screen.

In software, a basic latency counter records a start timestamp prior to invoking the RAG pipeline function, records a stop timestamp once the generated string is returned, and calculates the difference (`stop_time - start_time`). However, robust latency evaluation requires addressing several critical technical considerations.



Ten Critical Considerations for Latency Measurement

- 1. Prefer Latency Distributions (P50, P95, P99) Over Means/Averages:** Latency measurements are captured across defined time windows (e.g., last 1 hour, last 24 hours, last 1 month). Evaluating only the arithmetic mean hides severe tail latency spikes experienced by outlier requests. We must generate a Probability Density Function (PDF) across collected latency data and extract percentile metrics:
 - **P50 (Median):** The latency threshold within which 50% of all user requests are fulfilled.
 - **P95 (95th Percentile):** The latency threshold within which 95% of all user requests are fulfilled.
 - **P99 (99th Percentile):** The latency threshold within which 99% of all user requests are fulfilled. Analyzing P95 and P99 tail latencies exposes bad user experiences that mean latency masks.
- 2. Component Level Latency Breakdown:** End-to-end latency (e.g., 4.5 seconds overall) must be decomposed into granular stage timings. For example, breaking 4.5 seconds down into 1.5 seconds for retrieval and 3.0 seconds for generation reveals where optimizations are needed. Stage breakdowns must isolate query embedding duration, vector database retrieval, re-ranking latency and LLM text generation.
- 3. Measure Time to First Token (TTFT) Separately:** Time to First Token (TTFT) measures the time elapsed from user request submission to the generation and display of the very first response token from the LLM. Streaming responses token by token improves user experience

(UX) by allowing users to read output progressively rather than staring at a static loading state.

4. **Watch for Cold Starts:** System cold starts cause substantial latency spikes during initial requests. Cold starts occur due to server spin ups, initializing vector database connections, downloading or loading neural models (e.g., fetching cross-encoder re-rankers from Hugging Face), network handshakes, cache initializations, and serverless container instantiation. To prevent cold starts from corrupting latency evals, we must configure warm up runs (e.g., discarding the first 2 queries) and collect latency data starting from the 3rd execution onward.
5. **Track Token Count and Context Size Alongside Latency:** Latency scales directly with input context length and output token counts. Larger prompt contexts increase prompt processing time, while longer answer lengths require more autoregressive generation steps.
6. **Distinguish Latency from Throughput:** Latency measures single request processing duration, whereas throughput measures the system's parallel request processing capacity over time. Analogously, latency is the waiting time a customer experiences for their meal at a restaurant table, whereas throughput is the total number of customers the restaurant kitchen can serve per hour. When concurrent demand exceeds throughput capacity, requests are queued, causing latency spikes for queued users.
7. **Repeat Runs to Mitigate External API Noise:** External LLM APIs introduce operational noise due to fluctuating remote server loads. A single query run is statistically unreliable. We must repeat each test query multiple times (e.g., 5 repeated iterations per question across a sample set) and average the results to smooth out API variance.
8. **Track Failures and Timeouts Separately:** Latency cannot be analyzed in isolation. A test run might report an artificially low P95 latency if long running queries timed out and were excluded from the distribution. Timeout rates must be logged alongside latency metrics to ensure apparent speed improvements are not artifacts of dropped requests.
9. **Define Latency Budgets (Service Level Objectives / SLOs):** Engineering teams must establish clear Service Level Objectives (SLOs) for both system level and component level latencies. For example, an application SLO might dictate that end-to-end P95 latency must not exceed 3.0 seconds (3000 ms) and P95 TTFT must not exceed 1.2 seconds (1200 ms).
10. **Utilize Representative and Segmented Workloads:** Evaluation query sets must reflect realistic usage distributions rather than uniform simple prompts. A robust test suite should segment query difficulty (e.g., 3 simple questions, 4 medium questions, and 3 complex questions) to capture representative latency profiles.

Latency Evaluation Script (`eval_latency.py`) and Results Analysis

To implement these considerations, we construct an evaluation script (`eval_latency.py`). The script evaluates 5 representative questions, executes 5 repeat runs per question (25 total API calls), discards the first 2 initial runs as warm-up iterations, enables response streaming in `src/generator.py` to capture TTFT, and asserts SLO thresholds (P95 End-to-End \$le

3000\text{ ms}\\$, P95 TTFT \$\le 1200\text{ ms}\\$).

LATENCY EVALUATION REPORT					
Metric Stage	Mean	P50 (Med)	P95	P99	Min
Max					
End-to-End Latency	3.6s	3.8s	5.3s	5.3s	1.3s
5.3s					
└─ Retrieval Stage	756ms	762ms	983ms	1.0s	--
1.0s					
└─ Generation Stage	2.9s	--	--	--	--
--					
Time To First Token (TTFT)	1.6s	1.6s	2.0s	2.1s	1.3s
2.1s					
Average Answer Length: 1158 characters					
SLO Status: FAILED (End-to-End P95: 5.3s > 3.0s budget TTFT P95: 2.08s > 1.2s budget)					

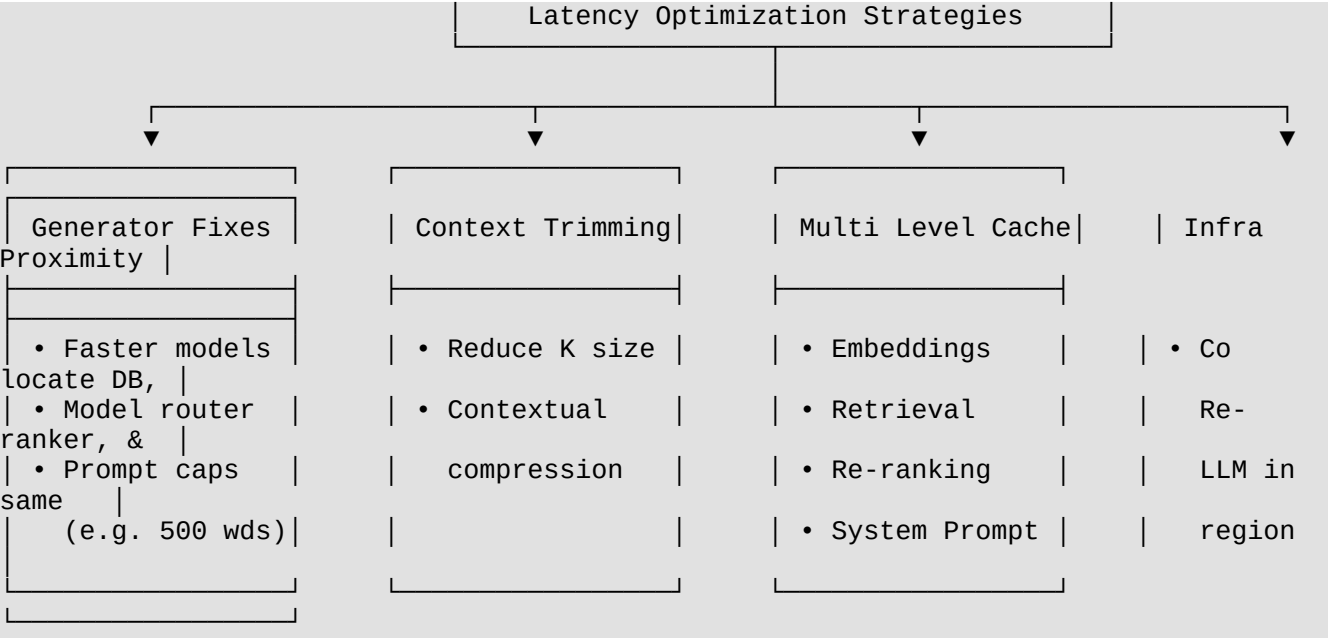
Detailed Breakdown of Latency Output:

- **End-to-End Latency:** Mean latency is 3.6 seconds, Median (P50) is 3.8 seconds, P95 latency is 5.3 seconds, P99 latency is 5.3 seconds, Minimum latency is 1.3 seconds, and Maximum latency is 5.3 seconds.
- **Stage Breakdown:** Retrieval averages ~700 ms (756 ms mean, 983 ms P95, 1.0 second max). Generation averages 2.9 seconds. The generator consumes nearly four times as much time as the retriever because retrieval runs locally while generation requires external LLM inference.
- **TTFT Performance:** Mean TTFT is 1.6 seconds, P50 is 1.6 seconds, P95 TTFT is 2.0 seconds (2081 ms), P99 TTFT is 2.1 seconds, Minimum TTFT is 1.3 seconds, and Maximum TTFT is 2.1 seconds.
- **Output Scale:** The average generated answer length across queries is 1158 characters.
- **SLO Evaluation:** The pipeline **FAILED both SLO targets**. The P95 end-to-end latency of 5.3 seconds breached the 3.0-second limit, and the P95 TTFT of 2.08 seconds breached the 1.2-second limit.

Engineering Strategies for Latency Reduction

When a pipeline fails its latency SLOs, engineering teams can apply several latency reduction techniques:

--	--



1. **Generator Optimizations:**

- **Switch to Faster Model Variants:** Replace heavy foundation models with high speed lightweight variants (e.g., migrating from Gemini 3.1 Pro to Gemini 3.1 Flash).
- **Implement a Model Router:** Deploy a query classification router that sends simple questions to a lightweight model and reserves larger models for complex queries.
- **Enforce System Prompt Constraints:** Direct the LLM to output concise responses and set maximum length caps (e.g., limiting responses to 500 words).

2. **Context Size Trimming:**

- **Reduce Retrieval Count (\$k\$):** Lower retrieved chunk volume (e.g., reducing (\$k\$) from 10 to 5).
- **Apply Contextual Compression:** Compress retrieved document passages before passing them to the prompt context.

3. **Sub Component Retrieval Optimization:**

- Isolate and optimize query embedding generation, vector database search, and cross-encoder re-ranking individually.

4. **Multi Level Caching:**

- **Embedding Caching:** Store generated embeddings for frequent queries.
- **Retrieval Caching:** Cache retrieved contexts for recurring user questions.
- **Re-ranking Caching:** Store re-ranked context ordering for repeated query passage combinations.
- **System Prompt Caching:** Cache static system instructions at the LLM provider level to

avoid redundant processing on every call.

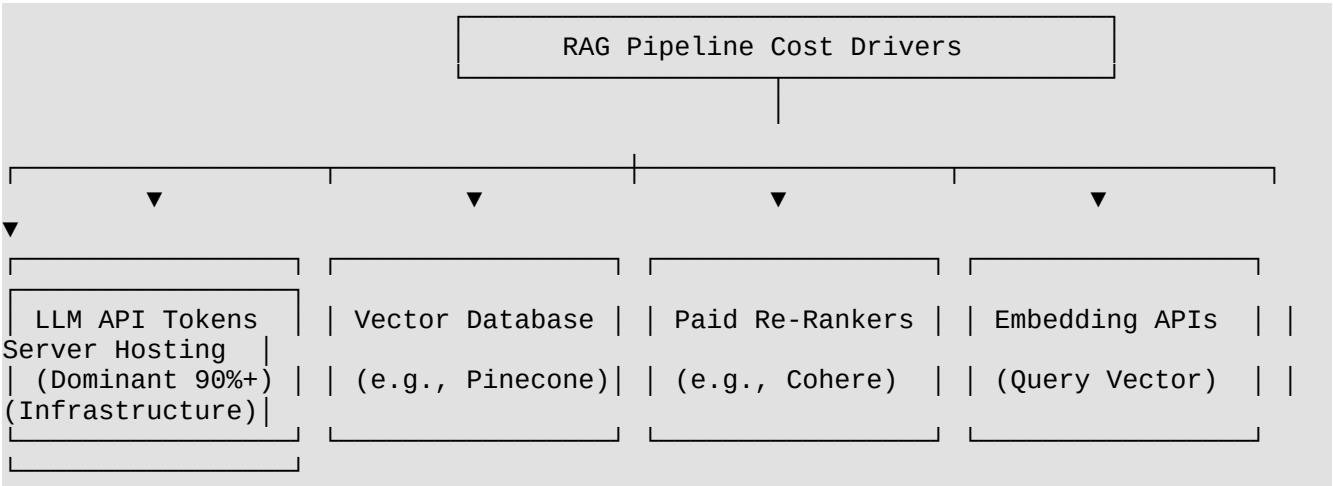
5. Infrastructure Proximity Optimization:

- Eliminate cross continental network latency hops. If serving users in India, host the vector database, re-ranker API (e.g., Cohere), and LLM API endpoints within the same geographical region (e.g., India) rather than spanning India, the US, and Europe.

5. Cost Evaluation

Definition and Cost Avenues in RAG Pipelines

Cost is defined as the monetary expense incurred to process and respond to a user query. Running a production RAG application involves five primary cost drivers:



In our setup, the vector database, re-ranker, and embedding models run locally or on free tiers, so our cost analysis focuses on LLM token burn, which represents the primary cost driver in production LLM applications.

LLM Token Pricing Mechanics and Calculations

LLM vendors charge for API usage based on consumed input and output tokens, priced per 1 Million (1M) tokens. Output tokens are significantly more expensive than input tokens typically costing approximately **four times as much** per token.

$$\text{Total LLM Cost} = (\text{Input Tokens} \times \text{Input Rate per Token}) + (\text{Output Tokens} \times \text{Output Rate per Token})$$

Key Considerations for Cost Measurement

1. **Measure Cost Per Query:** While tracking overall daily or monthly expenditure is useful, the primary operational metric is the monetary cost incurred per query processed.
2. **Separate Input and Output Costs:** Input prompt tokens (system rules, user query, retrieved context) and output tokens (generated text) must be logged independently to identify optimization opportunities.
3. **Analyze Cost Distributions (P95/P99):** Evaluating mean query cost alone hides outlier queries that burn excessive tokens. For instance, while an average query might cost 0.02 INR (2 paise), complex queries with extensive contexts might cost 1.50 INR or 3.00 INR. Long tail cost distributions reveal these expensive edge cases.
4. **Segment Cost by Query Complexity:** Track cost metrics separately across simple, medium, and complex query categories.
5. **Establish Cost Budgets and SLOs:** Translate business spending caps into per query operational budgets (e.g., enforcing a budget ceiling of 0.50 INR per query).

Cost Evaluation Script (eval_cost.py) and Results Analysis

We implement a cost evaluation script (eval_cost.py) to evaluate GPT-4o-mini pricing (\$0.15 per 1M input tokens, \$0.60 per 1M output tokens) across 4 questions executed over 3 repeat runs (12 total samples), applying a USD-to-INR conversion rate of ~95 INR/USD.

```
=====
COST EVALUATION REPORT
=====
Model Evaluated: GPT-4o-mini | Input Rate: $0.15/1M tokens | Output Rate: $0.60/1M tokens
Samples Processed: 12 execution runs across 4 test queries
-----
Token Consumption Averages:
  | Average Input Tokens: 1753 tokens (1109 tokens automatically prompt-cached by provider)
  | Average Output Tokens: 209 tokens
Monetary Cost Metrics:
  | Average Cost Per Query: $0.00021 USD (~0.02 INR / 2 paise)
  | Minimum Query Cost: $0.00019 USD
  | Maximum Query Cost: $0.00023 USD
Financial Projections (at 2,000 queries/day volume):
  | Daily Spend: ~57.00 INR
  | Monthly Spend: ~1,700.00 INR
-----
SLO Status: PASSED (Average cost of 0.02 INR is well within the 0.50 INR per query budget ceiling)
=====
```

Detailed Breakdown of Cost Output:

- **Token Consumption Breakdown:** Each query averaged 1753 input tokens and 209 output tokens. OpenAI automatically cached 1109 input tokens per query because identical system prompts and context structures were detected across repeated test runs.
 - **Per-Query Expenditure:** Average query cost was **\$0.00021 USD (~0.02 INR or 2 paise)**. The tight range between minimum cost (\$0.00019) and maximum cost (\$0.00023) demonstrates that query cost remains stable compared to latency.
 - **Input vs. Output Cost Dynamics:** In our Q&A chatbot, input costs outweigh output costs because retrieved prompt contexts are large relative to concise answers. Conversely, applications like coding agents display opposite dynamics—accepting minimal input prompts but generating extensive code outputs that drive up generation costs.
 - **Financial Projections:** Processing 2,000 queries per day results in an estimated cost of ~57 INR/day, translating to ~1,700 INR/month.
 - **SLO Evaluation:** The current system **PASSED its Cost SLO**, operating safely below our defined cost threshold.
-

Engineering Strategies for Cost Reduction

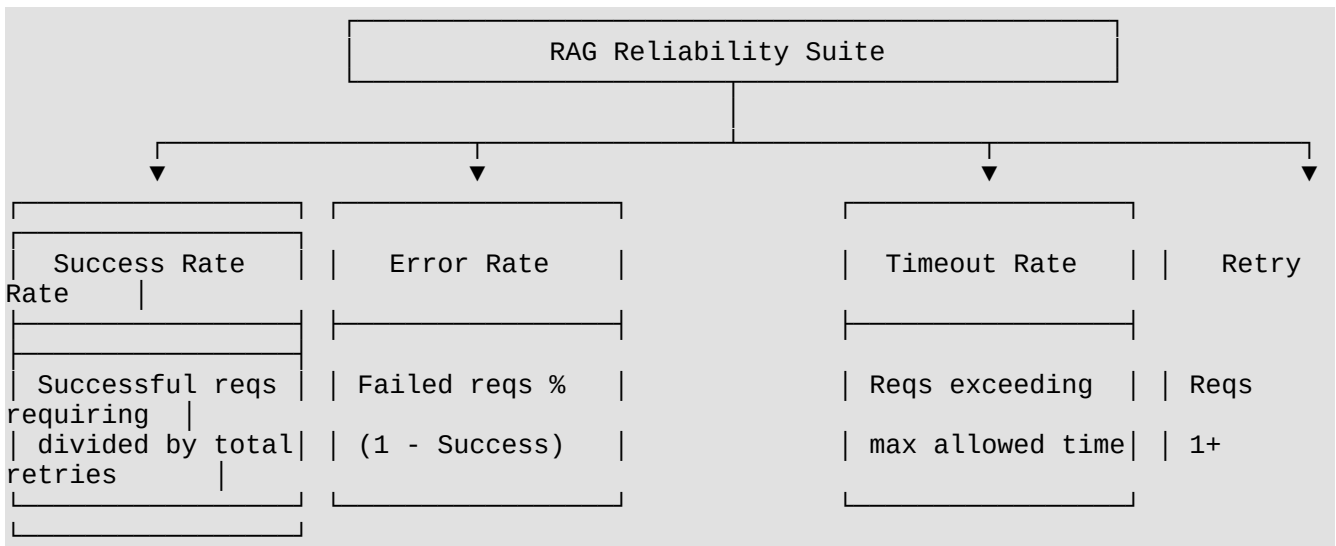
If a pipeline breaches its cost SLOs, engineering teams can apply several cost-reduction levers:

1. **Reduce Context Size:** Use smaller chunk sizes or contextual compression to decrease billable input tokens.
 2. **Prune System Prompts:** Streamline static instructions (e.g., trimming a system prompt from 1000 tokens down to 800 tokens without altering system logic).
 3. **Enforce Output Conciseness:** Instruct the LLM to provide shorter responses, directly reducing expensive output token generation.
 4. **Migrate to Cheaper or Open-Source Models:** Model selection serves as the most effective lever for cost reduction. At scale, self hosting open source LLMs on dedicated infrastructure can lower token costs compared to commercial APIs.
 5. **Leverage Aggressive Prompt Caching:** Structure prompt templates to maximize provider-side prompt caching for repeated context elements.
-

6. Reliability Evaluation

Definition and Core Metrics

Reliability is defined as the ability of a RAG pipeline to consistently serve user requests without errors, timeouts, crashes, or stage failures.



The four core reliability metrics are:

1. **Success Rate**: The percentage of user requests successfully processed and answered.
2. **Error Rate**: The percentage of user requests that fail ($\text{Error Rate} = 1 - \text{Success Rate}$).
3. **Timeout Rate**: The percentage of requests that exceed the maximum allowed execution time.
4. **Retry Rate**: The percentage of requests that require one or more retry attempts to complete.

Critical Considerations for Reliability Measurement

1. **Categorize Failures Granularly**: Reporting a single aggregate error rate is insufficient. Failures must be categorized using try-except exception handling across pipeline stages:
 - **LLM API Failures**: Remote service downtime or connection drops.
 - **Retriever / Vector DB Crashes**: Vector database failure or context retrieval errors.
 - **Re-ranker Failures**: External re-ranking API timeouts or errors.
 - **Timeout Errors**: Execution exceeding defined latency limits.
 - **Rate Limit Errors**: Exceeding vendor API requests-per-minute (RPM) or tokens-per-minute (TPM) quotas.
 - **Parser / Formatting Errors**: Failures when parsing structured outputs (e.g., invalid JSON).
 - **Internal Code Exceptions**: Software bugs triggered within application logic.
2. **Measure Reliability Under Concurrency / Load**: Single user local tests frequently show a 100% success rate. However, real-world production load with thousands of concurrent requests triggers connection pool exhaustion, rate limits, and pipeline crashes.

- 3. **Require Sufficiently Large Sample Sizes:** Small test sets (e.g., 20–50 requests) conceal low-frequency edge-case errors. Evaluating reliability accurately requires executing larger query volumes (e.g., 1000+ requests).
- 4. **Segment Workloads across Edge Cases:** Testing must cover simple queries, long-context queries, complex queries, long-answer queries, and structural edge cases.

Reliability Evaluation Script (eval_reliability.py) and Results Analysis

Our reliability evaluation script (eval_reliability.py) evaluates 4 questions executed across 5 repeat runs (20 total API requests), permitting a maximum retry limit of 2 retries per request.

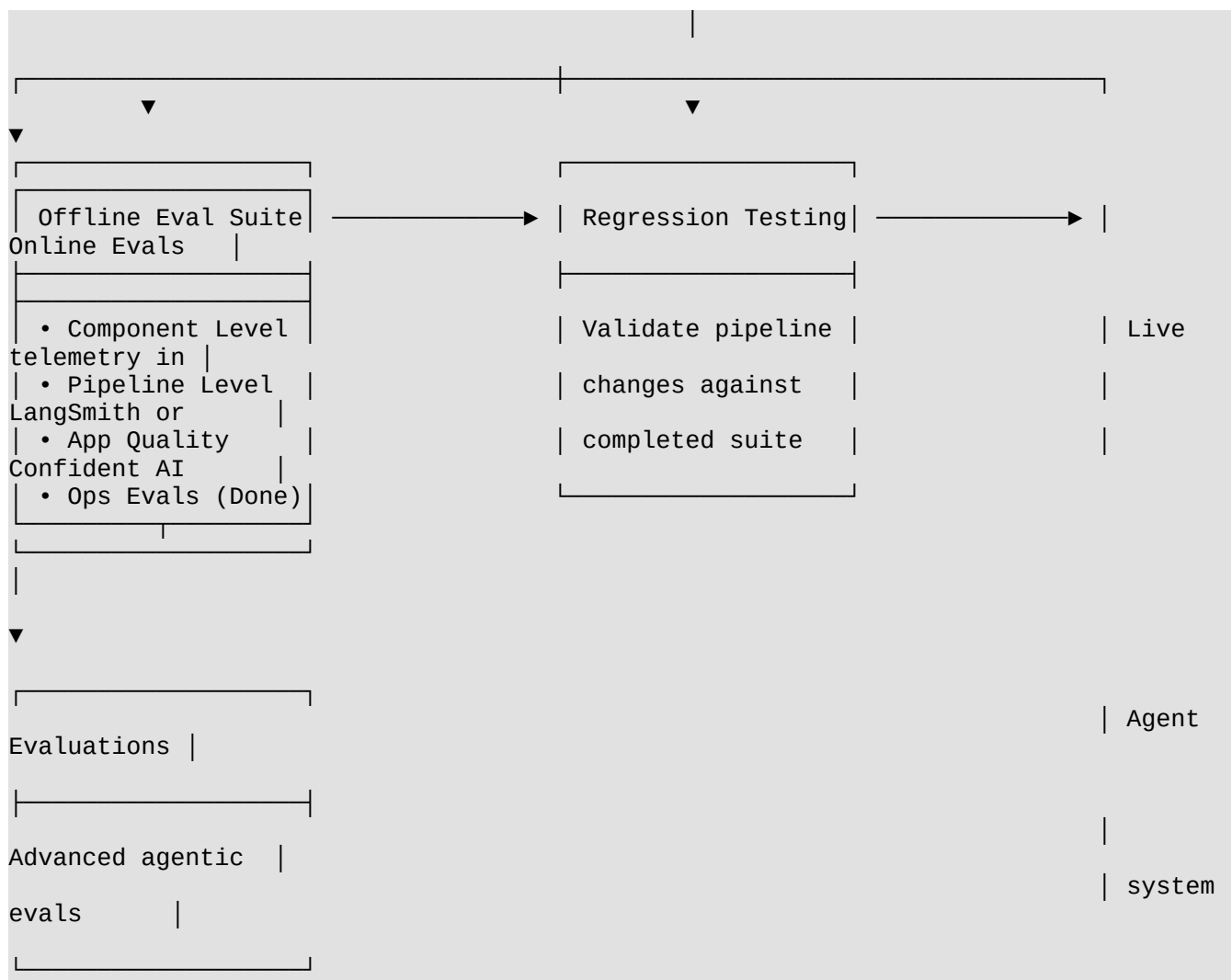
```
=====
                                RELIABILITY EVALUATION REPORT
=====
Execution Scope: 20 API requests processed across 4 test queries (Max Retries
Allowed = 2)
-----
Primary Reliability Scores:
├─ Success Rate: 100.0% (20/20 requests completed successfully)
├─ Error Rate:    0.0% (0/20 requests encountered unhandled failures)
├─ Retry Rate:   0.0% (0/20 requests required a retry attempt)
-----
Environment Context Note: Evaluated sequentially on a single dev laptop against a
commercial API.
Production Caution: Real-world concurrency or local self-hosted models (e.g.,
Ollama instances) will
yield lower reliability baseline scores under load.
=====
=====
```

Detailed Breakdown of Reliability Output:

- **Outcome:** The evaluation yielded a 100% Success Rate, a 0% Error Rate, and a 0% Retry Rate.
- **Baseline Context:** While a local sequential test against a commercial API yields ideal metrics, deploying under concurrent production loads or using local self-hosted models (e.g., cloud Ollama instances) will introduce rate limits, timeouts, and higher error rates that require ongoing monitoring.

7. Summary and Strategic Roadmap





With the implementation of Latency, Cost, and Reliability evaluations, our offline RAG evaluation suite is complete across all four structural domains: Component Level, Pipeline Level, Application Quality/Safety, and Operations.